BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

The Problem of Combinatorial Explosion:
If the environment has many possible states, and the agent operates for a long time, the table becomes impossibly large.

Chess has roughly 1040 legal board states. The number of possible game sequences is estimated at 10120 (The Shannon Number). There is no hard drive in the universe large enough to store a lookup table for Chess. No human could write it, and no compiler could build it.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Behaviors are dictated by Condition-Action Rules. In the code following lines represent the Condition-Action Rules. No processing of perceptions. Just reflexes mapped directly. (Hard coded code).

```
def sense_and_act(self, percept):
    # IF food_here THEN stay (collect)
    if percept.get('food_here'):
        return 'Stay'
    # IF wall_ahead or toxin_ahead THEN turn/move left
    if percept.get('wall_ahead') or percept.get('toxin_ahead'):
        return 'Left'
    # ELSE move 'Up' (a fixed forward direction for this simple agent)
    return 'Up'
```

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Since we cut-off all the global coordinates and the environment became partially observable. Simple reflex agents will only work if the environment is Fully Observable. Since a simple reflex agent only decides what to do from the current percept it can only decide what to do according to the condition action rules. Lack precept history make the agent run on a loop because it doesn't store the cells that it has already visited.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Transition Model (how the world evolves):

The agent never sees whether it actually moved, no position feedback exists. It infers this from percept changes across calls.

```
if blocked and self.last_action is not None:
    self.tried_directions.add(self.last_action)
elif not blocked:
    self.tried_directions.clear()
```

Still blocked after an action -> that action didn't produce a real transition, so it's remembered as failed. Blocked clears -> agent infers it successfully moved into a new state, so old failure history is discarded (it no longer applies).

Sensor Model (how actions affect perception):

wall_ahead/toxin_ahead are relative to self.facing, and execute_action updates facing to match whatever direction was just attempted. So each action doesn't just move the agent, it re-aims what gets sensed next. That's why memory is stored as directions tried, not coordinates: under partial observability, position isn't visible, only local

Finalize and Lock Lab 02 Documentation

FACULTY OF COMPUTING